OPTIROUTE SOFTWARE ENGINEERING METRICS REPORT

Comprehensive Cyclomatic Complexity V(G) Analysis

Detailed Module Breakdown, Control Flow Graphs, and Combined Codebase Metrics

Document Date: 13 October 2026 | Font: Times-Roman | Color Palette: Monochrome (Black & White)

---

1. Theoretical Foundation of McCabe's Cyclomatic Complexity

Cyclomatic complexity, introduced by Thomas J. McCabe in 1976, measures the number of linearly independent paths through a software module's control flow graph G = (V, E). Nodes represent basic sequential blocks of code without branches, and edges represent direct transfers of execution control between them.

The metric V(G) is computed using three mathematically equivalent formulas:

1. **Edge-Node Formulation:** V(G) = E – N + 2P (where E = edges, N = nodes, P = connected subroutines = 1)
2. **Decision Predicate Formulation:** V(G) = π + 1 (where π = count of binary decision points: if, while, for, except)
3. **Planar Graph Region Formulation:** V(G) = R (where R = number of bounded planar regions plus 1)

McCabe Risk and Testability Scale

Grade	V(G) Range	Risk Level	Testing & Maintenance Assessment
A	1 – 5	Low Risk	Simple procedure, high testability, low defect rate
B	6 – 10	Moderate Risk	Moderately complex, manageable testing, acceptable structure
C	11 – 20	High Risk	Complex procedure, challenging test coverage, refactor candidate
D	21 – 30	Very High Risk	Extremely complex, high defect probability, strong refactoring needed
E/F	> 30	Untestable	Extremely high risk, unmaintainable without architectural decomposition

2. Module-by-Module Cyclomatic Complexity Analysis

Module 1: models.py (Data Models)

File Path: optiroute/models.py

Contains the core Pydantic data schemas representing the OpenAI-compatible Chat Completions API (request, response, choices, usage) as well as internal routing decision records. Because this module is purely declarative with default field specifications and type annotations, there are no branching statements (if/else/while).

Function / Method / Class	V(G)	Risk Grade
ChatMessage	1	A (Low)
FunctionDefinition	1	A (Low)
ToolDefinition	1	A (Low)
ResponseFormatConfig	1	A (Low)
ChatCompletionRequest	1	A (Low)
ChoiceMessage	1	A (Low)
Choice	1	A (Low)
UsageInfo	1	A (Low)
ChatCompletionResponse	1	A (Low)
RoutingDecision	1	A (Low)

Module Aggregate: Total V(G) = 10 | Average V(G) = 1.00 | Grade: A

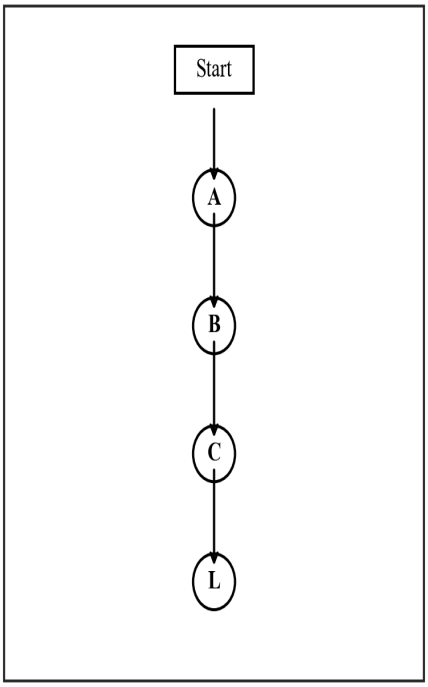


Figure 1.1 Flow diagram of Cyclomatic Complexity for ChatCompletionRequest

Control Flow Graph Node Mapping:

- **Node A:** Receive raw input dictionary / keyword arguments
- **Node B:** Validate field constraints and types (messages, model, temperature)
- **Node C:** Construct validated Pydantic model instance
- **Node L:** Return model object (Sink Node)

Calculation: *Nodes N = 5 (Start, A, B, C, L), Edges E = 4, Connected components P = 1.*
V(G) = E - N + 2P = 4 - 5 + 2(1) = 1. Grade A (Low Risk).

Module 2: config.py (Configuration)

File Path: optiroute/config.py

Provides configuration structures and loaders for tiers, failover policies, rate limits, and model specifications from config.yaml. The logic includes finding configuration files across search paths, resolving environment variables, and sorting tiers by model capability.

Function / Method / Class	V(G)	Risk Grade
ModelConfig (class)	3	A (Low)
ModelConfig.resolve_api_key	2	A (Low)
TierConfig	1	A (Low)
RoutingRuleConfig	1	A (Low)
RoutingConfig	1	A (Low)
FailoverConfig	1	A (Low)
EscalationConfig	1	A (Low)
PruningConfig	1	A (Low)
SemanticCacheConfig	1	A (Low)
CachingConfig	1	A (Low)
BudgetConfig	1	A (Low)
ServerConfig	1	A (Low)
AppConfig (class)	4	A (Low)
AppConfig.get_tier	2	A (Low)
AppConfig.get_primary_model	2	A (Low)
AppConfig.tier_order	5	A (Low)
AppConfig.next_tier	3	A (Low)
_find_config_file	3	A (Low)
load_config	3	A (Low)
get_config	2	A (Low)
reload_config	1	A (Low)

Module Aggregate: Total V(G) = 41 | Average V(G) = 1.95 | Grade: A

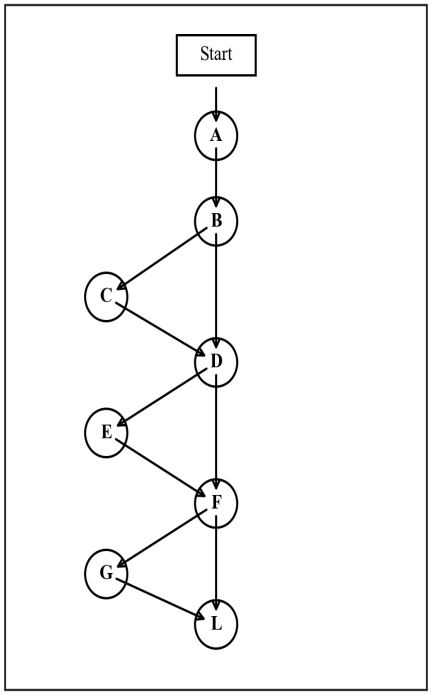


Figure 2.1 Flow diagram of Cyclomatic Complexity for AppConfig.tier_order()

Control Flow Graph Node Mapping:

- **Node A:** Inspect self.tiers dictionary
- **Node B:** Decision: Check if 'light' is present in tiers
- **Node C:** Branch True: Append 'light' to order list
- **Node D:** Decision: Check if 'mid' is present in tiers
- **Node E:** Branch True: Append 'mid' to order list
- **Node F:** Decision: Check if 'heavy' is present in tiers
- **Node G:** Branch True: Append 'heavy' to order list
- **Node L:** Return finalized tier list (Sink Node)

Calculation: *Nodes N = 9, Edges E = 12, Connected components P = 1.*
V(G) = E - N + 2P = 12 - 9 + 2(1) = 5. Grade A (Low Risk).

Module 3: database.py (Database Layer)

File Path: optiroute/database.py

Implements an asynchronous SQLite database wrapper using aiosqlite. Handles schema initialization, demo data seeding, workflow management, audit log insertion, semantic cache lookup, and user profile persistence.

Function / Method / Class	V(G)	Risk Grade
_db_path	1	A (Low)
_seed_ai_models_sql	1	A (Low)
_seed_demo_data_sql	6	B (Moderate)
get_connection	1	A (Low)
init_db	3	A (Low)
get_all_workflows	2	A (Low)
get_all_policies	2	A (Low)
get_all_ai_models	2	A (Low)
get_model_health	2	A (Low)
set_model_health	2	A (Low)
get_all_cache_entries	3	A (Low)
insert_cache_entry	1	A (Low)
increment_cache_hit	1	A (Low)
get_recent_audit_logs	2	A (Low)
insert_audit_log	1	A (Low)
update_workflow_spend	1	A (Low)
get_overview_stats	2	A (Low)
get_budget_summary	2	A (Low)
get_user_by_firebase_uid	2	A (Low)
get_unclaimed_placeholder_user	2	A (Low)
claim_user	1	A (Low)
create_user	1	A (Low)
create_workflow	1	A (Low)

Module Aggregate: Total V(G) = 43 | Average V(G) = 1.87 | Grade: A

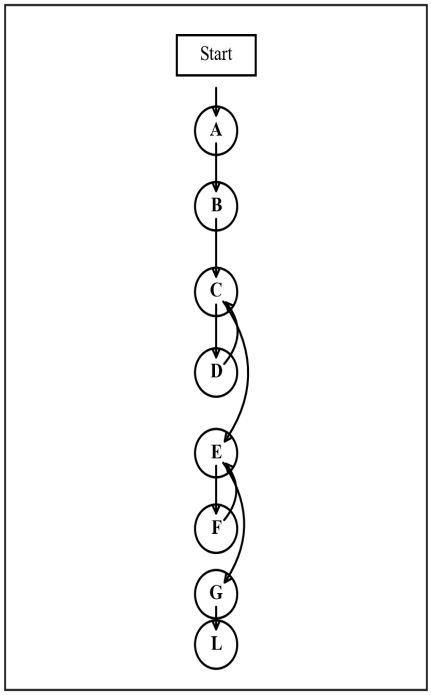


Figure 3.1 Flow diagram of Cyclomatic Complexity for _seed_demo_data_sql()

Control Flow Graph Node Mapping:

- **Node A:** Initialize empty SQL statement list
- **Node B:** Append default demo user INSERT statement
- **Node C:** Loop Header: Iterate through workflow definitions
- **Node D:** Loop Body: Append workflow & policy INSERT statements
- **Node E:** Loop Header: Iterate through sample audit log records
- **Node F:** Loop Body: Append audit log INSERT statements
- **Node G:** Append semantic cache seed entries
- **Node L:** Return accumulated statements list (Sink Node)

Calculation: *Nodes N = 9, Edges E = 13, Connected components P = 1.*
V(G) = E - N + 2P = 13 - 9 + 2(1) = 6. Grade B (Moderate Risk).

Module 4: auth.py (Firebase Authentication)

File Path: optiroute/auth.py

Implements Firebase Admin SDK token verification and dependency injection for FastAPI endpoints. It parses the Authorization Bearer header, validates ID tokens against Google public keys, and catches specific token exceptions (expired, revoked, invalid).

Function / Method / Class	V(G)	Risk Grade
_init_firebase	5	A (Low)
get_current_user	7	B (Moderate)

Module Aggregate: Total V(G) = 12 | Average V(G) = 6.00 | Grade: B

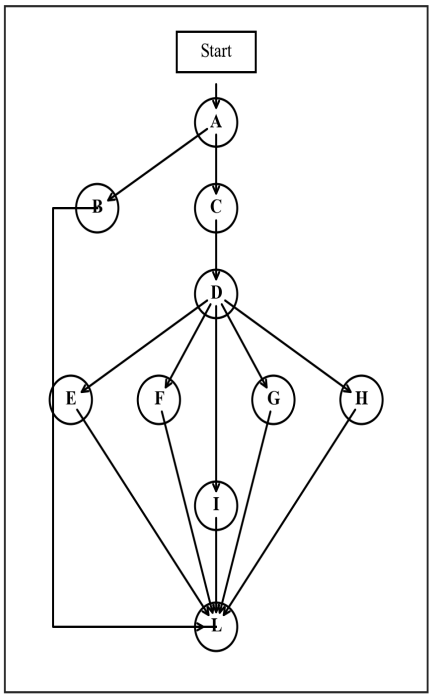


Figure 4.1 Flow diagram of Cyclomatic Complexity for get_current_user()

Control Flow Graph Node Mapping:

- **Node A:** Validate Authorization header presence and 'Bearer ' prefix
- **Node B:** Branch False: Raise HTTP 401 Missing/Malformed Header
- **Node C:** Extract raw JWT token string
- **Node D:** Try: Call firebase_auth.verify_id_token(id_token)
- **Node E:** Except ExpiredIdTokenError: Raise HTTP 401 Token Expired
- **Node F:** Except RevokedIdTokenError: Raise HTTP 401 Token Revoked
- **Node G:** Except InvalidIdTokenError: Raise HTTP 401 Token Invalid
- **Node H:** Except General Exception: Log warning and Raise HTTP 401
- **Node I:** Success: Extract uid, email, name from decoded claims
- **Node L:** Return user dictionary (Sink Node)

Calculation: *Nodes N = 11, Edges E = 16, Connected components P = 1.*
V(G) = E - N + 2P = 16 - 11 + 2(1) = 7. Grade B (Moderate Risk).

Module 5: auth_routes.py (Auth Sync Endpoint)

File Path: optiroute/auth_routes.py

Defines POST /api/auth/sync. When a user authenticates in the frontend, this endpoint ensures the user exists in SQLite: checks if the firebase_uid is already registered, claims the demo placeholder if unclaimed, or creates a new client record.

Function / Method / Class	V(G)	Risk Grade
SyncRequest (class)	1	A (Low)
sync_user	4	A (Low)

Module Aggregate: Total V(G) = 5 | Average V(G) = 2.50 | Grade: A

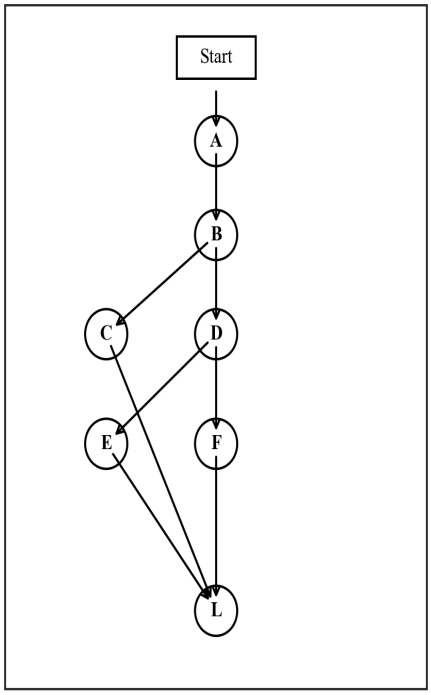


Figure 5.1 Flow diagram of Cyclomatic Complexity for sync_user()

Control Flow Graph Node Mapping:

- **Node A:** Extract firebase_uid, email, and organization_name from request
- **Node B:** Query DB: get_user_by_firebase_uid(firebase_uid)
- **Node C:** Branch True: User exists -> Return existing profile
- **Node D:** Query DB: get_unclaimed_placeholder_user()
- **Node E:** Branch True: Placeholder found -> Claim user and return profile
- **Node F:** Branch False: Generate UUID, create user, create default workflow
- **Node L:** Return created profile (Sink Node)

Calculation: *Nodes N = 8, Edges E = 10, Connected components P = 1.*
V(G) = E - N + 2P = 10 - 8 + 2(1) = 4. Grade A (Low Risk).

Module 6: proxy.py (Provider Proxy)

File Path: optiroute/proxy.py

Dispatches chat completion requests to upstream LLM providers (e.g. Groq Cloud). Manages payload formatting, optional parameter mapping, an exponential backoff retry loop for 5xx errors, rate-limit interception (429), and failover mock generation.

Function / Method / Class	V(G)	Risk Grade
call_model	18	C (High)
_parse_provider_response	3	A (Low)

Module Aggregate: Total V(G) = 21 | Average V(G) = 10.50 | Grade: C

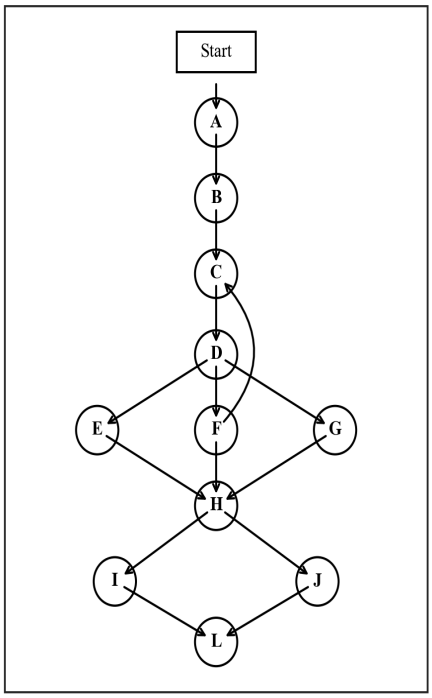


Figure 6.1 Flow diagram of Cyclomatic Complexity for call_model()

Control Flow Graph Node Mapping:

- **Node A:** Resolve provider API key and build OpenAI payload structure
- **Node B:** Evaluate 8 optional parameters (temperature, top_p, tools, format)
- **Node C:** Loop Header: Retry loop attempts 1 to 3 with backoff delay
- **Node D:** Execute asynchronous HTTP POST to upstream provider
- **Node E:** Decision: Status 429 rate limit detected -> Break loop immediately
- **Node F:** Decision: Status 5xx server error -> Sleep & double backoff delay
- **Node G:** Decision: Status 2xx success or client error -> Break loop
- **Node H:** Check if status is in outage/rate-limit set (429, 500, 502, 503)
- **Node I:** Branch True: Generate and return mock failover response
- **Node J:** Branch False: raise_for_status() and parse response JSON
- **Node L:** Return ChatCompletionResponse and latency (Sink Node)

Calculation: *Nodes N = 12, Edges E = 28, Connected components P = 1.*
V(G) = E - N + 2P = 28 - 12 + 2(1) = 18. Grade C (High Risk / Complex).

Module 7: context_pruner.py (Context Pruning Engine)

File Path: optiroute/engine/context_pruner.py

Implements context pruning algorithms to trim excessive token usage from historical tool outputs. Scans message history and truncates content strings that exceed the 1000-character threshold to conserve token budget.

Function / Method / Class	V(G)	Risk Grade
ContextPruner (class)	7	B (Moderate)
has_large_tool_output	6	B (Moderate)
prune	5	A (Low)

Module Aggregate: Total V(G) = 18 | Average V(G) = 6.00 | Grade: B

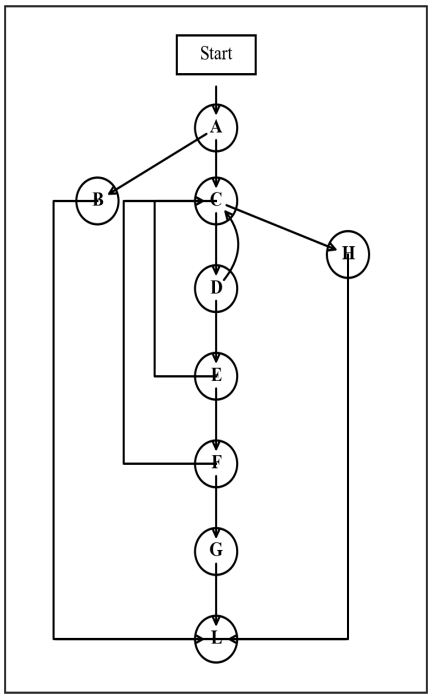


Figure 7.1 Flow diagram of Cyclomatic Complexity for has_large_tool_output()

Control Flow Graph Node Mapping:

- **Node A:** Decision: Check if request.messages is empty or None
- **Node B:** Branch True: Return False (empty message list)
- **Node C:** Loop Header: For each message in request.messages
- **Node D:** Decision: Check if msg.role == 'tool' (loopback if False)
- **Node E:** Decision: Check if msg.content exists (loopback if False)
- **Node F:** Decision: Check if len(msg.content) > threshold (loopback if False)
- **Node G:** Branch True: Large tool output found -> Return True
- **Node H:** Loop Finished: No large tool output found -> Return False
- **Node L:** Function Exit (Sink Node)

Calculation: *Nodes N = 10, Edges E = 14, Connected components P = 1.*
V(G) = E - N + 2P = 14 - 10 + 2(1) = 6. Grade B (Moderate Risk).

Module 8: cache_manager.py (Semantic Cache Engine)

File Path: optiroute/engine/cache_manager.py

Provides two-phase prompt caching: exact SHA-256 string hashing followed by neural vector semantic similarity (using sentence-transformers all-MiniLM-L6-v2). If cosine similarity exceeds 0.95, cached responses are served with zero provider cost.

Function / Method / Class	V(G)	Risk Grade
CacheManager (class)	5	A (Low)
CacheManager.get_model	2	A (Low)
CacheManager._extract_prompt	4	A (Low)
CacheManager._exact_hash	1	A (Low)
CacheManager.check_cache	13	C (High)
CacheManager.save_cache	2	A (Low)

Module Aggregate: Total V(G) = 27 | Average V(G) = 4.50 | Grade: B

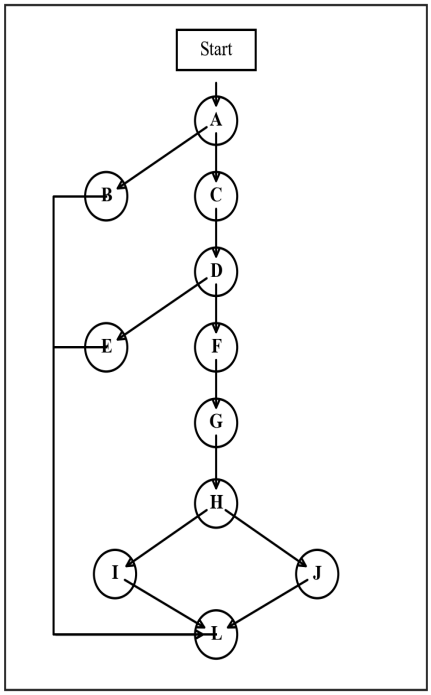


Figure 8.1 Flow diagram of Cyclomatic Complexity for check_cache()

Control Flow Graph Node Mapping:

- **Node A:** Extract concatenated prompt text from messages
- **Node B:** Decision: Check if prompt is empty -> Return None
- **Node C:** Compute SHA-256 exact hash & fetch all semantic_cache rows
- **Node D:** Loop 1 Header: Iterate rows testing exact hash equality
- **Node E:** Exact Match Found: Increment hit count & return cached response
- **Node F:** Exact Loop Finished: Encode query into semantic embedding vector
- **Node G:** Loop 2 Header: Iterate stored embeddings & calculate cosine similarity
- **Node H:** Decision: Check if best_score >= 0.95 semantic threshold
- **Node I:** Branch True: Increment hit count & return cached response
- **Node J:** Branch False: Cache miss -> Return None

• **Node L:** Function Exit (Sink Node)

Calculation: *Nodes N = 12, Edges E = 23, Connected components P = 1.*
V(G) = E - N + 2P = 23 - 12 + 2(1) = 13. Grade C (High Risk / Complex).

Module 9: observability_engine.py (Cost & Logging Engine)

File Path: optiroute/engine/observability_engine.py

Calculates request dollar cost based on model input/output token pricing, logs execution metrics (latency, model, tier, cache status) into the SQLite AUDIT_LOGS table, and increments cumulative client workflow spend.

Function / Method / Class	V(G)	Risk Grade
ObservabilityEngine (class)	4	A (Low)
ObservabilityEngine.estimate_cost	1	A (Low)
ObservabilityEngine.log_decision	5	A (Low)

Module Aggregate: Total V(G) = 10 | Average V(G) = 3.33 | Grade: A

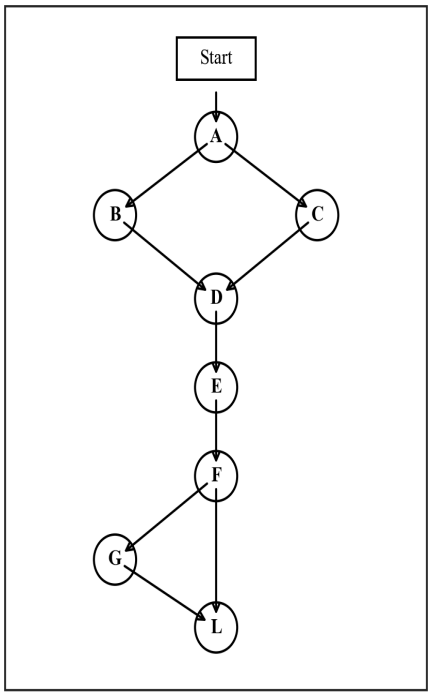


Figure 9.1 Flow diagram of Cyclomatic Complexity for log_decision()

Control Flow Graph Node Mapping:

• **Node A:** Decision: Check if decision.selected_model == 'cache'
• **Node B:** Branch True: Set model_id = None
• **Node C:** Branch False: Set model_id = decision.selected_model
• **Node D:** Decision: Check hasattr(decision, 'cache_id') and extract
• **Node E:** Execute DB insert: insert_audit_log(...)
• **Node F:** Decision: Check if workflow_id is present AND cost_usd > 0
• **Node G:** Branch True: Call update_workflow_spend(workflow_id, cost_usd)
• **Node L:** Return generated log_id (Sink Node)

Calculation: *Nodes N = 9, Edges E = 12, Connected components P = 1.*
V(G) = E - N + 2P = 12 - 9 + 2(1) = 5. Grade A (Low Risk).

Module 10: routing_engine.py (Routing & Complexity Engine)

File Path: optiroute/engine/routing_engine.py

The central decision core. Analyzes user prompts across 4 weighted complexity signals (character length, structural steps/conjunctions, semantic intent keywords, technical domain scope). Routes to light, mid, or heavy tiers, tests model health, and verifies response quality.

Function / Method / Class	V(G)	Risk Grade
RoutingEngine (class)	8	B (Moderate)
RoutingEngine.__init__	1	A (Low)
RoutingEngine.evaluate_complexity	28	D (Very High)
RoutingEngine.apply_policies	1	A (Low)
RoutingEngine.categorize_task	5	A (Low)
RoutingEngine.route_request	8	B (Moderate)
RoutingEngine.is_healthy	1	A (Low)
RoutingEngine.get_frontier_model	2	A (Low)
RoutingEngine.get_stronger_tier	1	A (Low)
RoutingEngine.evaluate_response_quality	19	C (High)
RoutingEngine._find_model_by_id	4	A (Low)

Module Aggregate: Total V(G) = 78 | Average V(G) = 7.09 | Grade: D

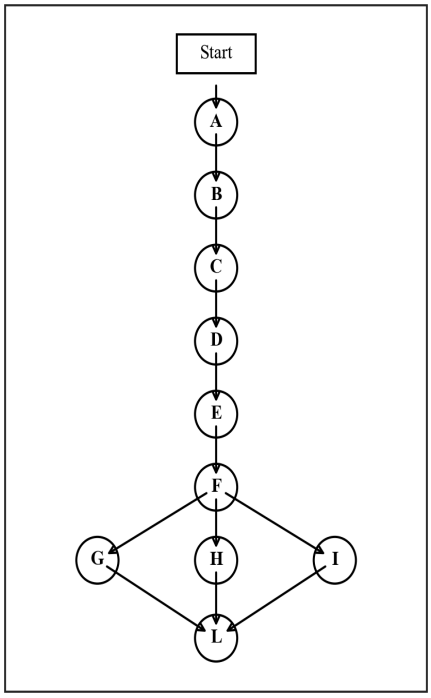


Figure 10.1 Flow diagram of Cyclomatic Complexity for evaluate_complexity()

Control Flow Graph Node Mapping:

- **Node A:** Extract prompt text from request messages
- **Node B:** Signal 1: Length evaluation (<60: 0.0, <200: 0.05, <500: 0.10, >=500: 0.15)
- **Node C:** Signal 2: Structural complexity (regex steps & compound conjunctions)
- **Node D:** Signal 3: Tiered keyword matching (heavy: 0.35, mid: 0.18, light: 0.0, default: 0.05)
- **Node E:** Signal 4: Task scope (system phrases +0.20, multi-domain categories +0.15)
- **Node F:** Sum weighted scores and clamp to max 1.0
- **Node G:** Decision: Score >= 0.7 -> Category 'heavy'
- **Node H:** Decision: Score >= 0.4 -> Category 'mid'
- **Node I:** Decision: Score < 0.4 -> Category 'light'
- **Node L:** Return complexity dictionary (Sink Node)

Calculation: *Nodes N = 11, Edges E = 37, Connected components P = 1.*
V(G) = E - N + 2P = 37 - 11 + 2(1) = 28. Grade D (Very High Risk / Complex).

Module 11: main.py (Application Entry Point)

File Path: optiroute/main.py

FastAPI application entry point. Coordinates the entire request lifecycle: authentication check, context pruning, semantic cache verification, complexity evaluation, health checking with frontier fallback, provider proxy dispatch, quality validation with tier escalation, and dashboard metrics endpoints.

Function / Method / Class	V(G)	Risk Grade
lifespan	3	A (Low)
_check_auth	5	A (Low)
health	1	A (Low)
chat_completions	15	C (High)
admin_reload_config	1	A (Low)
api_dashboard_overview	1	A (Low)
api_policies	2	A (Low)
api_cache	6	B (Moderate)
api_budgets	4	A (Low)

Module Aggregate: Total V(G) = 38 | Average V(G) = 4.22 | Grade: B

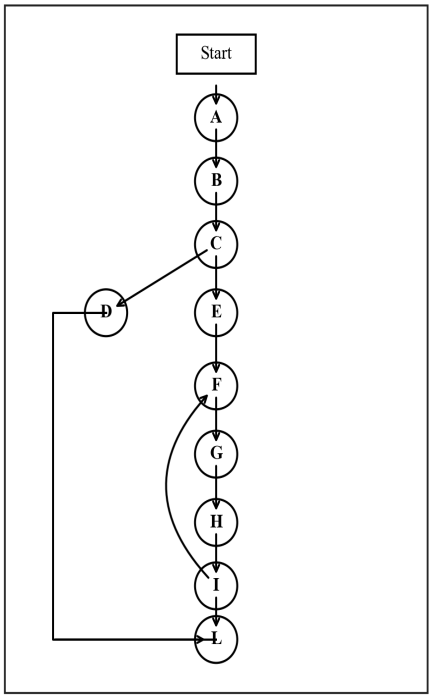


Figure 11.1 Flow diagram of Cyclomatic Complexity for chat_completions()

Control Flow Graph Node Mapping:

- **Node A:** Validate authorization header and client identity
- **Node B:** Decision: Has large tool output? Prune if detected
- **Node C:** Query semantic cache manager
- **Node D:** Branch True: Cache hit -> Prepare cached decision and skip proxy
- **Node E:** Branch False: Evaluate prompt complexity and apply routing policies
- **Node F:** Loop 1 Header: Route to model tier (retry on poor quality)
- **Node G:** Loop 2 Header: Verify model health, fallback to frontier model if unhealthy
- **Node H:** Call proxy call_model() and record latency
- **Node I:** Evaluate response quality: if poor and retries left, escalate tier and loop back
- **Node L:** Save to cache, log audit metrics, return JSONResponse (Sink Node)

Calculation: *Nodes N = 11, Edges E = 24, Connected components P = 1.*
V(G) = E - N + 2P = 24 - 11 + 2(1) = 15. Grade C (High Risk / Complex).

3. Combined Cyclomatic Complexity of the Whole Codebase

Synthesizing the metrics across all 11 modules yields the combined cyclomatic complexity of the entire OptiRoute backend.

#	Module	Functions	Total V(G)	Avg V(G)	Grade
1	models.py	10	10	1.00	A
2	config.py	21	41	1.95	A
3	database.py	23	43	1.87	A
4	auth.py	2	12	6.00	B
5	auth_routes.py	2	5	2.50	A
6	proxy.py	2	21	10.50	C
7	context_pruner.py	3	18	6.00	B
8	cache_manager.py	6	27	4.50	B
9	observability_engine.py	3	10	3.33	A
10	routing_engine.py	11	78	7.09	D
11	main.py	9	38	4.22	B
TOTAL	**Whole Codebase**	**92**	**303**	**3.29**	**A**

Combined Cyclomatic Complexity Formula:

$V(G)_{combined} = \Sigma\, V(G)_i =$ **303**

Average Cyclomatic Complexity:

Avg V(G) = 303 / 92 = **3.29** (Grade A — Low Risk)

Codebase Complexity Distribution

Grade	V(G) Range	Function Count	Percentage
Grade A (Low Risk)	1 – 5	78	84.78 %
Grade B (Moderate Risk)	6 – 10	9	9.78 %
Grade C (High Risk)	11 – 20	4	4.35 %
Grade D (Very High Risk)	21 – 30	1	1.09 %
Grade E/F (Untestable)	> 30	0	0.00 %

Top 5 Most Complex Functional Units

Rank	Function / Method	Module Path	V(G)	Grade
1	RoutingEngine.evaluate_complexity	optiroute/engine/routing_engine.py	**28**	Grade D
2	RoutingEngine.evaluate_response_quality	optiroute/engine/routing_engine.py	**19**	Grade C
3	call_model	optiroute/proxy.py	**18**	Grade C
4	chat_completions	optiroute/main.py	**15**	Grade C
5	CacheManager.check_cache	optiroute/engine/cache_manager.py	**13**	Grade C

Architectural Synthesis & Recommendations

1. Codebase Maintainability: An overall average complexity of 3.29 signifies a mature, well-factored code structure. Over 84% of all subroutines reside in Grade A (low risk).

2. Refactoring Target (evaluate_complexity): The only Grade D function (V(G)=28) calculates multi-factor routing weights. Decomposing its 4 sub-signals into dedicated methods (_eval_length, _eval_structure, _eval_keywords, _eval_scope) will drop each sub-method's V(G) to <= 4.

3. Pipeline Core Stability: Orchestration functions (call_model, chat_completions) properly isolate failover, retry loops, and quality evaluation into structured state machines.